

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Isha Venkataraman
Register Number	3122247001023
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To implement a single-layer Perceptron Learning Algorithm (PLA) from scratch and a Multilayer Perceptron (MLP) trained with backpropagation on the English Handwritten Characters dataset, tune the MLP hyperparameters step by step, and compare both models using accuracy, precision, recall, F1-score, confusion matrices, ROC curves and convergence curves.

2 Problem Statement

The task is to recognise handwritten characters from images. The input is a 32×32 grayscale image flattened into a 1,024-dimensional vector with pixel values in $[0, 1]$. The output is one of 62 classes (digits 0–9, uppercase A–Z and lowercase a–z). Model A is a single layer of perceptrons (PLA) and Model B is an MLP with a hidden layer and non-linear activation. Both are trained on the same data so that the difference in test performance can be credited to the model type and the tuning.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle (images in the <code>Img</code> folder, labels in <code>english.csv</code>)
Number of Samples	3,410 images (1200×900 RGB scans)
Number of Features	1,024 (32×32 pixels after preprocessing)
Number of Classes	62 (10 digits, 26 uppercase, 26 lowercase), 55 images per class
Missing Values	None (0 missing image paths, 0 missing labels, 0 duplicate file names)
Train-Test Split	Stratified. Train 2,294 (37 per class), Validation 434 (7 per class), Test 682 (11 per class)

4 Exploratory Data Analysis

The exploratory analysis covers the dataset overview, missing values, class distribution, pixel statistics, correlation and a PCA view of the data.

Dataset Overview and Preprocessing Example

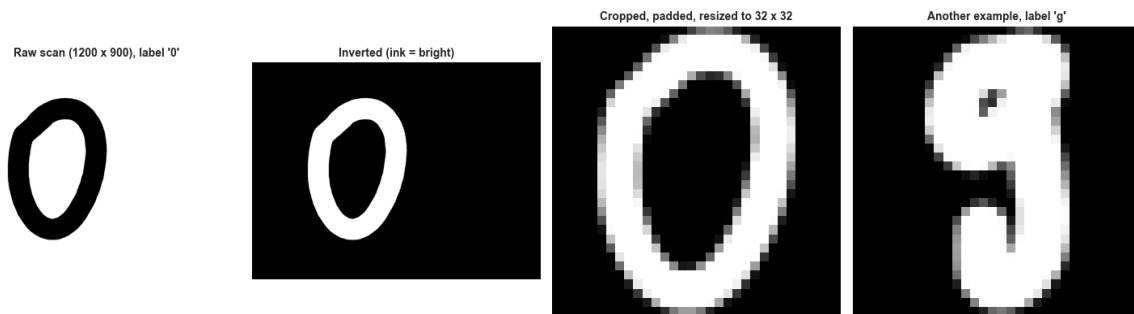


Figure 1: Raw scan, inverted image and the final 32×32 image for the same sample, plus another example (label g)

Inference: The raw scan is large and mostly white background, so it cannot be fed to the models directly. After inversion the ink is bright and the cropping step removes the empty area around the character. The padded square canvas keeps the aspect ratio, so the character is not stretched when it is resized to 32×32 .

Missing Value Analysis

All 3,410 image paths and labels are present, every file exists on disk and there are no duplicate file names. So no imputation or removal of samples was needed.

Class Distribution

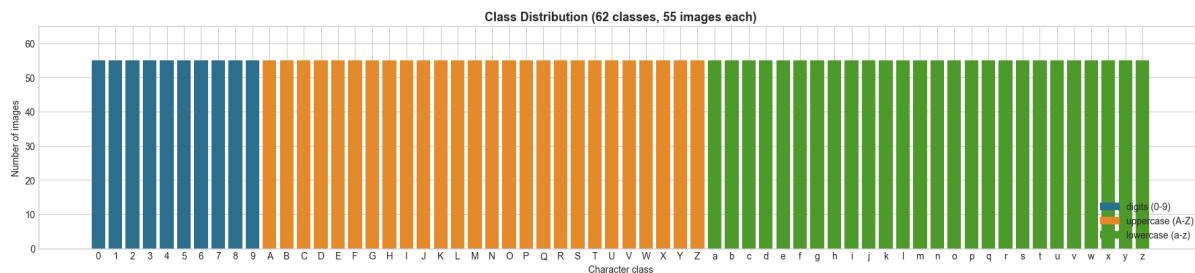


Figure 2: Class distribution of the 62 classes (55 images each)

Inference: Every class has exactly 55 images, so the dataset is perfectly balanced. Because of this, accuracy and macro-averaged metrics are almost the same thing here, and no resampling or class weights were required. The 62 classes split into 10 digits, 26 uppercase and 26 lowercase letters.

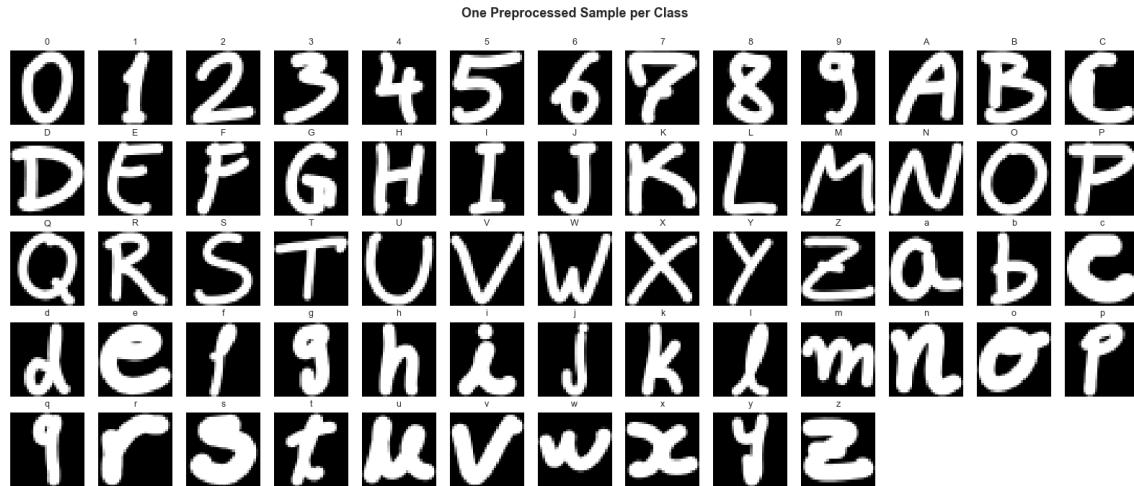


Figure 3: One preprocessed sample for each of the 62 classes

Inference: The samples show that the preprocessing gives centred, similarly sized characters for all classes. Many pairs look alike after resizing, for example 0/O/o, 1/I/l and S/s. Such pairs are expected to be the main source of errors for any model that only sees pixels.

Statistical Summary

Table 2: Per-image statistics by class group

Feature	Statistic	Digit	Lower	Upper
Pixel mean	count	550	1430	1430
	mean	0.285	0.313	0.289
	std	0.078	0.098	0.076
	min	0.113	0.096	0.095
	25%	0.231	0.247	0.235
	50%	0.283	0.308	0.287
	75%	0.334	0.379	0.341
	max	0.577	0.617	0.513
Pixel std	count	550	1430	1430
	mean	0.420	0.429	0.420
	std	0.037	0.042	0.036
	min	0.295	0.276	0.277
	25%	0.399	0.409	0.398
	50%	0.426	0.439	0.426
	75%	0.448	0.463	0.448
	max	0.479	0.483	0.481
Ink fraction	count	550	1430	1430
	mean	0.320	0.345	0.327
	std	0.081	0.102	0.080
	min	0.133	0.115	0.108
	25%	0.261	0.277	0.271
	50%	0.320	0.340	0.326
	75%	0.372	0.415	0.384
	max	0.606	0.661	0.554

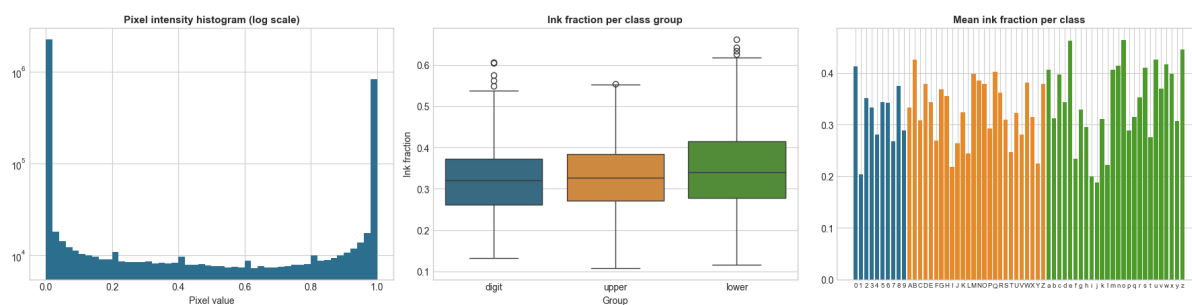


Figure 4: Pixel intensity histogram, ink fraction per class group and mean ink fraction per class

Inference: Most pixels are either close to 0 (background) or close to 1 (ink), which is why the histogram is drawn on a log scale. The three class groups have nearly the same pixel mean (0.285 to 0.313) and ink fraction (0.32 to 0.35), so simple intensity features cannot tell the groups apart. Lowercase letters have a slightly wider spread than the other two groups.

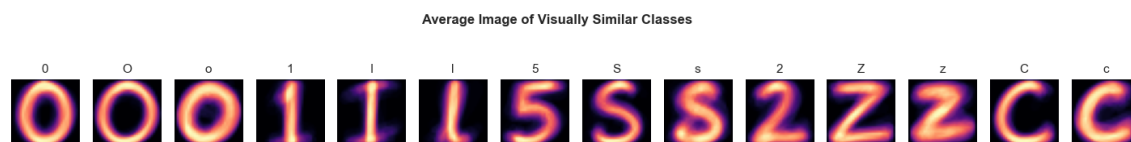


Figure 5: Average image of visually similar classes

Inference: The averaged images of 0, O and o, and of 1, I and l, look almost identical after normalisation. This means the classifiers will find these groups very hard to separate. It also gives an early idea of where most of the confusions in the results will come from.

Correlation Matrix

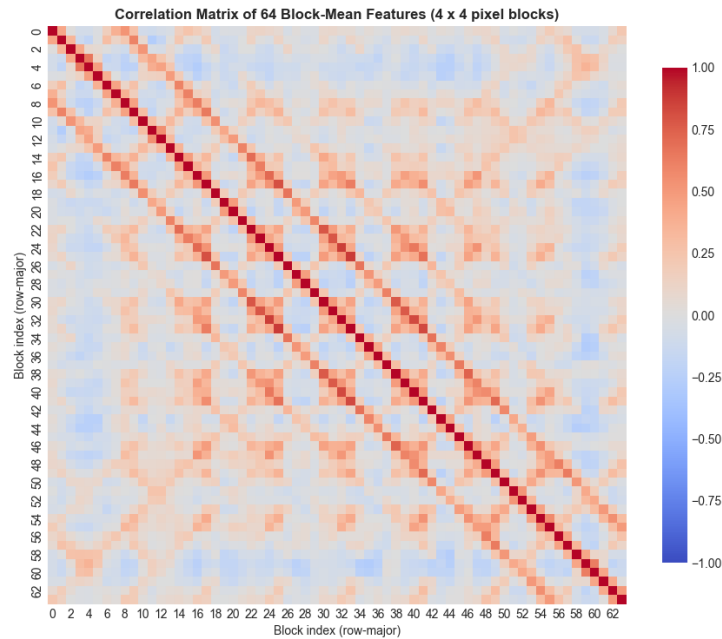


Figure 6: Correlation matrix of 64 block-mean features (each block is 4×4 pixels)

Inference: The red diagonal bands show that blocks next to each other are correlated. Most of the other pairs are close to zero: only 3.4% of the block pairs have $|r| > 0.5$ and the mean $|r|$ is 0.138. So the raw pixels contain a lot of redundancy locally, but there is no single direction that carries most of the information.

PCA View of the Data

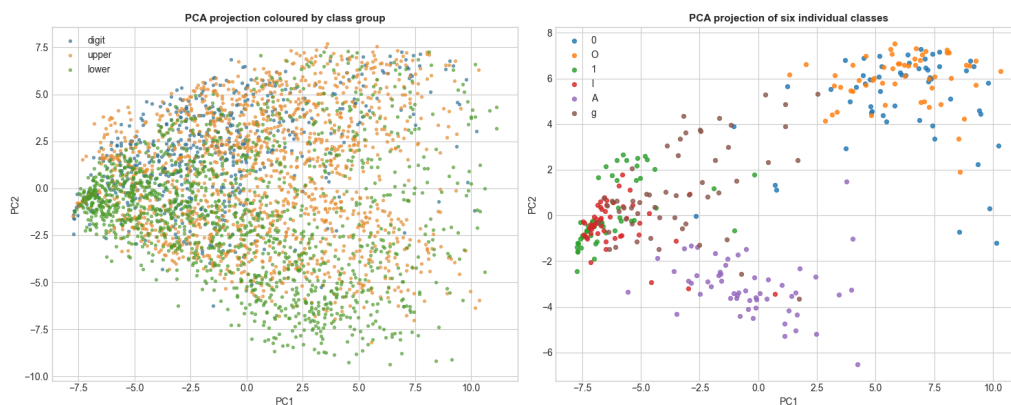


Figure 7: PCA projection coloured by class group (left) and for six individual classes (right)

Inference: The first 2, 10 and 50 principal components explain 19.4%, 51.1% and 85.8% of the variance. In the 2D projection the classes lie on top of each other, including the individual classes 0, O, 1, l, A and g. A set of straight lines will therefore not be able to separate all 62 classes, which points towards a non-linear model.

5 Data Preprocessing

- **Missing value handling:** No missing values were found, so nothing was imputed.
- **Image processing:** Each image was converted to grayscale and inverted so that the ink is bright. A threshold of 50 was used to find the bounding box of the character. The crop was placed on a square canvas with 8 pixels of padding, resized to 32×32 with area interpolation and divided by 255. The image is then flattened to a 1,024-dimensional vector.
- **Encoding:** The 62 character labels were converted to integers 0–61 with `LabelEncoder`. One-hot targets are used for the PLA and the MSE loss.
- **Feature scaling:** Pixel values were scaled to $[0, 1]$. No other scaling was applied.
- **Train-test split:** A stratified 80/20 split gives a test set of 682 images. The remaining 2,728 images were split again into 2,294 training and 434 validation images (37 and 7 per class). The validation set is used for tuning and early stopping and the test set is used only once at the end. Both models use exactly the same split.
- **Feature engineering:** None was used for the models. PCA and block-mean features were used only for visualisation.

6 Machine Learning Algorithm

Model A: Perceptron Learning Algorithm (PLA)

Working principle. The PLA uses one perceptron per class (one-vs-rest), so there are 62 units with 1,025 weights each (1,024 inputs and a bias). Each unit gives 1 or 0 with a step activation. For every training sample, the units that give a wrong output are corrected. The predicted class is the unit with the largest net input. Since the data is not linearly separable, the weights from the epoch with the best validation accuracy are kept (pocket idea).

Mathematical formulation.

$$\hat{y} = \begin{cases} 1 & \mathbf{w}^T \mathbf{x} + b > 0 \\ 0 & \text{otherwise} \end{cases}, \quad \mathbf{w}_{t+1} = \mathbf{w}_t + \eta (y - \hat{y}) \mathbf{x} \quad (1)$$

Advantages. It is simple, very fast to train (3.3 s here) and needs no gradients.

Limitations. It only gives good results for linearly separable data, never settles when the data overlaps, has no margin and gives only 0/1 outputs.

Model B: Multilayer Perceptron (MLP)

Working principle. The MLP has an input layer of 1,024 units, one or more hidden layers with a non-linear activation and an output layer of 62 units. The weights are learned with backpropagation and a mini-batch optimizer. The hidden layer learns features from the pixels and the output layer combines them, which allows curved decision boundaries.

Mathematical formulation.

$$\mathbf{h} = \phi(W_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{z} = W_2 \mathbf{h} + \mathbf{b}_2, \quad p_k = \frac{e^{z_k}}{\sum_j e^{z_j}} \quad (2)$$

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^{62} y_{ik} \log p_{ik} \quad (3)$$

$$\mathbf{v} \leftarrow \mu \mathbf{v} + \nabla_{\mathbf{w}} L, \quad \mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{v} \quad (4)$$

Here ϕ is the activation (ReLU, tanh or sigmoid), L is the cross-entropy loss, μ is the momentum and η is the learning rate. L2 weight decay adds a penalty on the weights.

Advantages. It can learn non-linear patterns, gives probabilities for every class and can be improved through tuning.

Limitations. It has many hyperparameters, needs more data to avoid overfitting, and works on flattened pixels so it does not use the 2D structure of the image.

7 Experimental Setup

Python Version	3.13.5
Scikit-Learn Version	
Deep Learning Library	PyTorch 2.10.0 (CPU, 10 threads)
IDE	Jupyter Notebook
Operating System	
Hardware	

8 Hyperparameter Selection

The MLP was tuned one factor at a time in seven stages, in this order: activation, optimizer and learning rate, batch size, architecture, loss function, regularization and data augmentation. In each stage the best value from the earlier stages was kept fixed. Every configuration was trained with three seeds and the mean validation accuracy was used to choose, so that one lucky run does not decide the result. The starting (untuned) configuration was one hidden layer of 256 ReLU units, Adam with $\eta = 0.001$, batch size 64, cross-entropy, no regularization, 60 epochs and patience 10.

Final Hyperparameters

Table 3: Final MLP hyperparameters

Parameter	Value
Hidden layers	(256,)
Activation	ReLU
Optimizer	SGD with momentum 0.9
Learning rate	0.05
Batch size	32
Loss function	Cross-entropy
Dropout	0.0
Weight decay (L2)	0.0001
Data augmentation	False
Max epochs / patience	150 / 20

Table 4: PLA settings

Parameter	Value
Units	62 (one per class, one-vs-rest)
Activation	Step
Learning rate η	1.0
Initial weights	Zero
Epochs	60
Model kept	Weights of best validation epoch (epoch 38)

PLA Learning Rate Check

With zero initial weights, changing η only scales all the weights by the same factor. The step function and the argmax do not change, so every η gives the same errors, as the table shows.

Table 5: PLA learning-rate check (zero initialisation, 20 epochs)

η	Train error (epoch 20)	Best val accuracy	Best epoch	Mean $ w $
0.01	0.0615	0.6866	18	0.0521
0.10	0.0615	0.6866	18	0.5212
1.00	0.0615	0.6866	18	5.2120
10.00	0.0615	0.6866	18	52.1203

Tuning Stage 1: Activation Function

ReLU was chosen. It has the highest mean validation accuracy, although tanh is almost the same, and it is cheaper to compute.

Table 6: Stage 1: activation function (3 seeds)

Config	Val acc (mean)	Val acc (std)	Train acc	Best epoch	Time (s)	Params	Gap
relu	0.7972	0.0050	0.9696	21.67	1.17	278334	0.1724
tanh	0.7965	0.0047	0.9788	24.33	1.29	278334	0.1823
sigmoid	0.7888	0.0115	0.9337	25.33	1.31	278334	0.1450

Tuning Stage 2: Optimizer and Learning Rate

Each optimizer was tried with its own range of learning rates. SGD with momentum and $\eta = 0.05$ was best, and it also had the smallest spread between seeds.

Table 7: Stage 2: optimizer $\times$ learning rate (3 seeds)

Config	Val acc (mean)	Val acc (std)	Train acc	Best epoch	Time (s)	Params	Gap
sgd, lr=0.01	0.7296	0.0078	0.7700	51.67	1.01	278334	0.0403
sgd, lr=0.05	0.7719	0.0068	0.8612	31.00	0.74	278334	0.0893
sgd, lr=0.1	0.7811	0.0132	0.9236	29.67	0.81	278334	0.1425
sgd, lr=0.3	0.8018	0.0065	0.9653	23.67	0.64	278334	0.1634
sgd_momentum, lr=0.005	0.7880	0.0075	0.9092	44.33	1.29	278334	0.1212
sgd_momentum, lr=0.01	0.7865	0.0054	0.9000	23.33	0.80	278334	0.1135
sgd_momentum, lr=0.05	0.8065	0.0019	0.9927	29.00	0.84	278334	0.1863
sgd_momentum, lr=0.1	0.7911	0.0076	0.9753	19.67	0.67	278334	0.1842
adam, lr=0.0001	0.7642	0.0043	0.8864	55.67	1.91	278334	0.1222
adam, lr=0.0005	0.7896	0.0066	0.9638	28.00	1.21	278334	0.1743
adam, lr=0.001	0.7972	0.0050	0.9696	21.67	1.03	278334	0.1724
adam, lr=0.005	0.7849	0.0039	0.9725	15.00	0.82	278334	0.1876

Tuning Stage 3: Batch Size

Batch size 32 was best. Batch size 16 was unstable with this learning rate, and very large batches gave fewer updates per epoch.

Table 8: Stage 3: batch size (3 seeds)

Config	Val acc (mean)	Val acc (std)	Train acc	Best epoch	Time (s)	Params	Gap
batch=16	0.5998	0.0093	0.6915	15.00	1.93	278334	0.0917
batch=32	0.8126	0.0071	0.9942	30.33	1.57	278334	0.1816
batch=64	0.8065	0.0019	0.9927	29.00	0.96	278334	0.1863
batch=128	0.7949	0.0019	0.9705	21.33	0.50	278334	0.1756
batch=256	0.7796	0.0022	0.8806	16.33	0.32	278334	0.1010

Tuning Stage 4: Hidden Layers and Neurons

One hidden layer of 256 units was the best. Making it wider did not help and all the deeper networks were worse.

Table 9: Stage 4: architecture (3 seeds)

Config	Val acc (mean)	Val acc (std)	Train acc	Best epoch	Time (s)	Params	Gap
(128,)	0.7849	0.0207	0.9741	23.67	1.04	139198	0.1892
(256,)	0.8126	0.0071	0.9942	30.33	1.54	278334	0.1816
(512,)	0.8103	0.0057	0.9711	13.67	1.14	556606	0.1608
(1024,)	0.8103	0.0022	0.9799	21.67	2.81	1113150	0.1697
(256, 128)	0.7581	0.0050	0.9108	19.00	1.32	303294	0.1527
(512, 256)	0.7957	0.0217	0.9725	25.33	2.40	672062	0.1768
(512, 256, 128)	0.7412	0.0054	0.8750	20.33	2.42	697022	0.1339
(256, 256, 256)	0.7512	0.0050	0.8845	18.00	2.03	409918	0.1333

Tuning Stage 5: Cost Function

Cross-entropy on the logits was compared with mean squared error on the softmax output. Cross-entropy gave better validation accuracy.

Table 10: Stage 5: loss function (3 seeds)

Config	Val acc (mean)	Val acc (std)	Train acc	Best epoch	Time (s)	Params	Gap
cross_entropy	0.8126	0.0071	0.9942	30.33	1.61	278334	0.1816
mse	0.7911	0.0029	0.9393	26.00	1.58	278334	0.1482

Tuning Stage 6: Regularization

Dropout and L2 weight decay were tried alone and together (up to 100 epochs). Only weight decay 10^{-4} gave a small gain, so it was selected.

Table 11: Stage 6: regularization (3 seeds, up to 100 epochs)

Config	Val acc (mean)	Val acc (std)	Train acc	Best epoch	Time (s)	Params	Gap
none	0.8134	0.0068	0.9939	34.33	1.90	278334	0.1805
dropout 0.2	0.7873	0.0029	0.9730	36.33	2.16	278334	0.1857
dropout 0.4	0.7089	0.0093	0.8330	24.33	1.65	278334	0.1241
weight decay 1e-4	0.8157	0.0050	0.9910	29.67	1.86	278334	0.1753
weight decay 1e-3	0.7980	0.0066	0.9823	26.00	1.67	278334	0.1843
dropout 0.2 + wd 1e-4	0.7919	0.0060	0.9661	38.67	2.21	278334	0.1743
dropout 0.4 + wd 1e-4	0.7243	0.0228	0.8333	25.67	1.75	278334	0.1091

Tuning Stage 7: Data Augmentation

Random rotation (up to 12 degrees), scaling (0.9 to 1.1) and shifts (up to 2 pixels) were applied every epoch. It reduced the train-validation gap but not the validation accuracy, so it was not used.

Table 12: Stage 7: data augmentation (3 seeds, up to 120 epochs)

Config	Val acc (mean)	Val acc (std)	Train acc	Best epoch	Time (s)	Params	Gap
no augmentation	0.8157	0.0050	0.9910	29.67	1.99	278334	0.1753
augmentation	0.8049	0.0132	0.8633	38.67	3.77	278334	0.0584

5-Fold Cross-Validation of the Final Configuration

The final configuration was also checked with stratified 5-fold cross-validation on the training plus validation data, to see if the chosen setting is stable.

Table 13: 5-fold cross-validation (mean accuracy = 0.7687 ± 0.0192)

Fold	Accuracy	Best epoch	Time (s)
1	0.7619	14	1.2730
2	0.7656	22	1.6386
3	0.7949	37	2.1564
4	0.7780	25	1.8540
5	0.7431	11	1.2265

9 Results

Precision, recall and F1-score are macro averages over the 62 classes. ROC-AUC is the macro average of the one-vs-rest curves.

Table 14: Test set metrics of PLA and the tuned MLP

Metric	PLA	MLP (tuned)
Accuracy	0.6437	0.7317
Precision	0.6651	0.7492
Recall	0.6437	0.7317
F1-score	0.6395	0.7305
ROC-AUC	0.9600	0.9847

Table 15: A/B comparison on the test set (Acc. = accuracy, Prec. = precision, Rec. = recall, wtd. = weighted)

Model	Acc.	Prec. (macro)	Rec. (macro)	F1 (macro)	F1 (wtd.)	AUC (micro)	AUC (macro)
PLA	0.6437	0.6651	0.6437	0.6395	0.6395	0.9588	0.9600
MLP (untuned)	0.7229	0.7388	0.7229	0.7218	0.7218	0.9858	0.9872
MLP (tuned)	0.7317	0.7492	0.7317	0.7305	0.7305	0.9841	0.9847

Table 16: A/B comparison: training accuracy, size and time

Model	Train accuracy	Parameters	Training time (s)
PLA	0.9908	63,550	3.3097
MLP (untuned baseline)	0.9738	278,334	0.9306
MLP (tuned)	0.9956	278,334	3.7850

Table 17: Mean F1 by class group

Group	PLA F1	MLP F1
digit	0.6675	0.7292
lower	0.5779	0.6968
upper	0.6905	0.7648

Table 18: Classes where the MLP gains the most over the PLA

Class	Group	PLA F1	MLP F1	Gain
g	lower	0.000	0.476	0.476
S	upper	0.250	0.636	0.386
r	lower	0.600	0.957	0.357
W	upper	0.500	0.833	0.333
n	lower	0.581	0.870	0.289
K	upper	0.400	0.667	0.267
C	upper	0.500	0.762	0.262
4	digit	0.500	0.750	0.250

Table 19: Hardest classes (lowest MLP F1)

Class	Group	PLA F1	MLP F1
l	lower	0.250	0.364
s	lower	0.357	0.381
I	upper	0.455	0.400
1	digit	0.667	0.414
g	lower	0.000	0.476
x	lower	0.476	0.476
o	lower	0.556	0.500
v	lower	0.632	0.556

Table 20: Most frequent PLA confusions

True	Predicted	Count
C	c	7
W	w	5
0	O	4
e	c	4
o	0	4
S	s	4
I	1	4
l	I	3
X	x	3
S	8	3

Table 21: Most frequent tuned MLP confusions

True	Predicted	Count
I	1	5
Z	z	4
v	V	4
o	0	4
p	P	4
g	9	4
1	l	4
l	1	3
C	c	3
w	W	3

10 Result Visualization

PLA Results

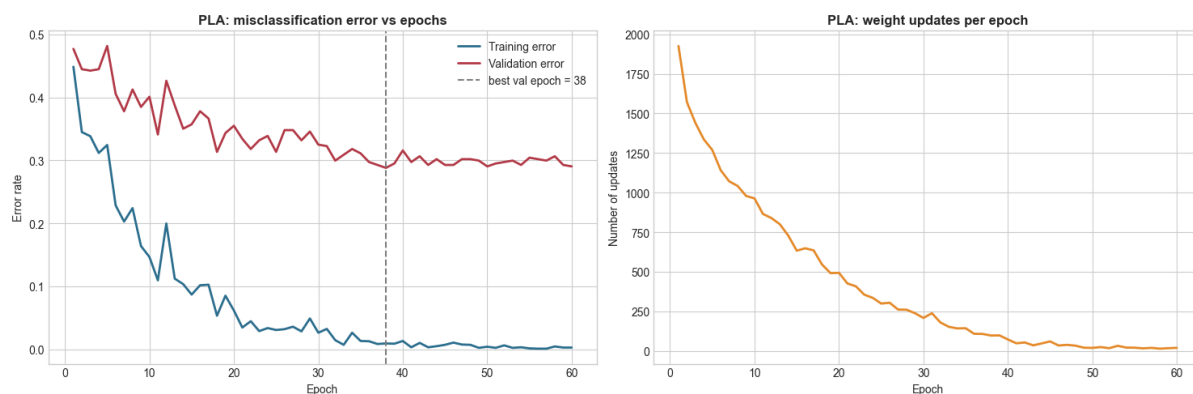


Figure 8: PLA misclassification error vs epochs (left) and weight updates per epoch (right)

Inference: The training error keeps falling and reaches 0.0026 at epoch 60, but the validation error stays between 0.29 and 0.35 and moves up and down. The number of updates per epoch drops from 1,926 to 19 but never reaches zero, so the algorithm did not fully converge. The best validation epoch was 38. This gap between training and validation error shows that the PLA is memorising the training set.

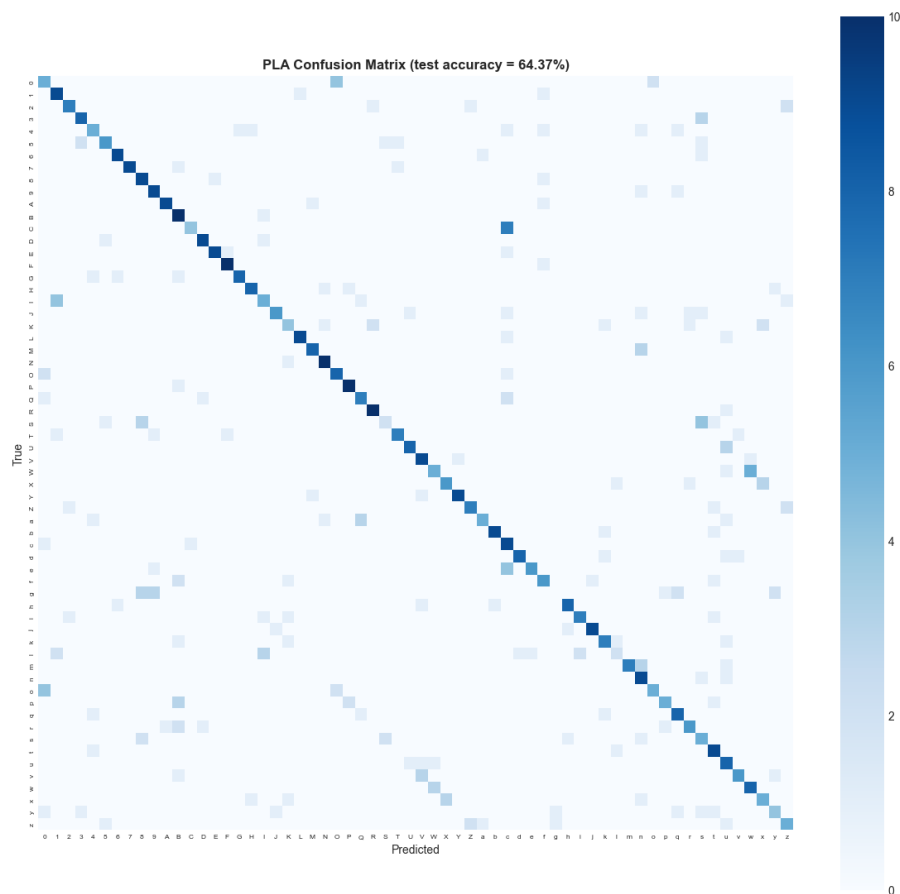


Figure 9: Confusion matrix of the PLA on the test set (accuracy 64.37%)

Inference: Most of the counts are on the diagonal, but there are many scattered off-diagonal entries. The largest confusions are C predicted as c (7 times), W as w (5) and 0 with O. These are case pairs and look-alike characters. The PLA also completely fails for the class g ($F1 = 0$).

MLP Tuning Results

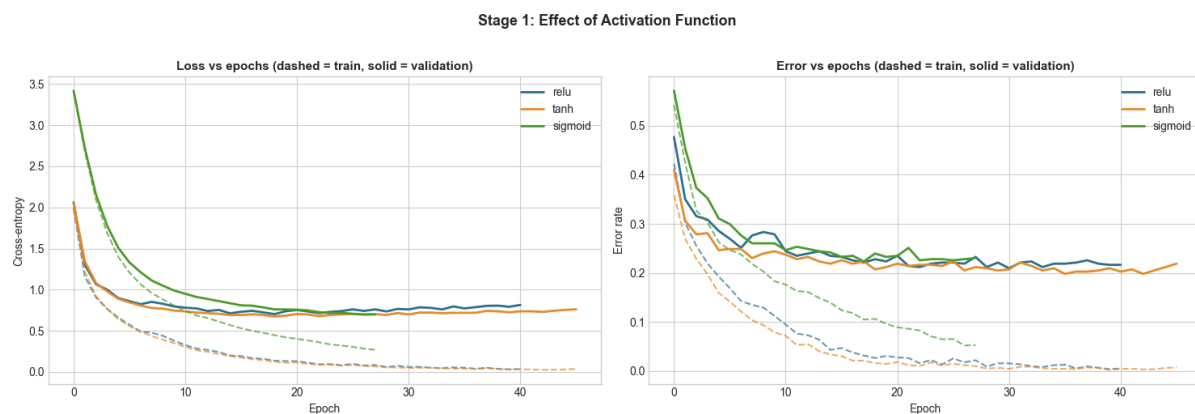


Figure 10: Stage 1: loss and error vs epochs for ReLU, tanh and sigmoid

Inference: ReLU and tanh reach almost the same validation accuracy (0.7972 and 0.7965), and sigmoid is a little lower (0.7888) with a higher spread between seeds. Sigmoid also fits the training data less (train accuracy 0.9337). Activation was one of the least important hyperparameters here. ReLU was picked because it is the simplest and the best by a small margin.

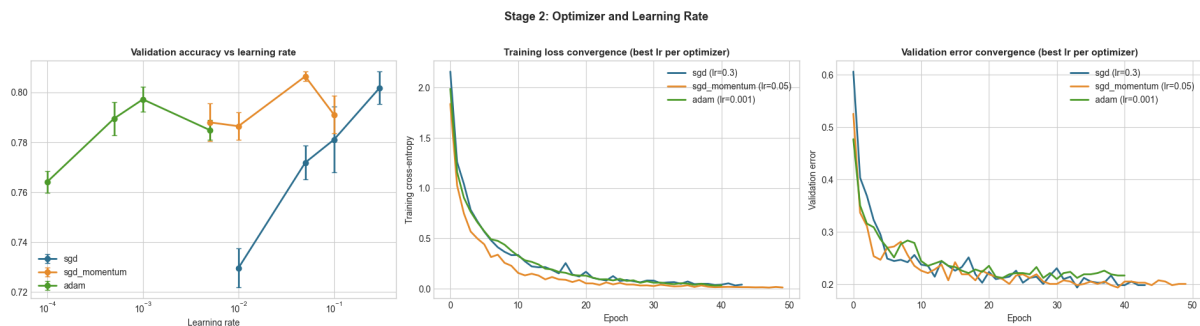


Figure 11: Stage 2: validation accuracy vs learning rate, training loss and validation error convergence for SGD, SGD with momentum and Adam

Inference: Validation accuracy ranged from 0.7296 (SGD, 0.01) to 0.8065 (SGD with momentum, 0.05). Adam reached its best result in about 22 epochs at $\eta = 0.001$, while plain SGD at 0.01 needed about 52 epochs and was 7 points lower. Adam was less sensitive to the learning rate, and plain SGD needed a learning rate 30 times larger (0.3) to catch up. SGD with momentum at 0.05 gave the best result with the smallest spread (0.0019).

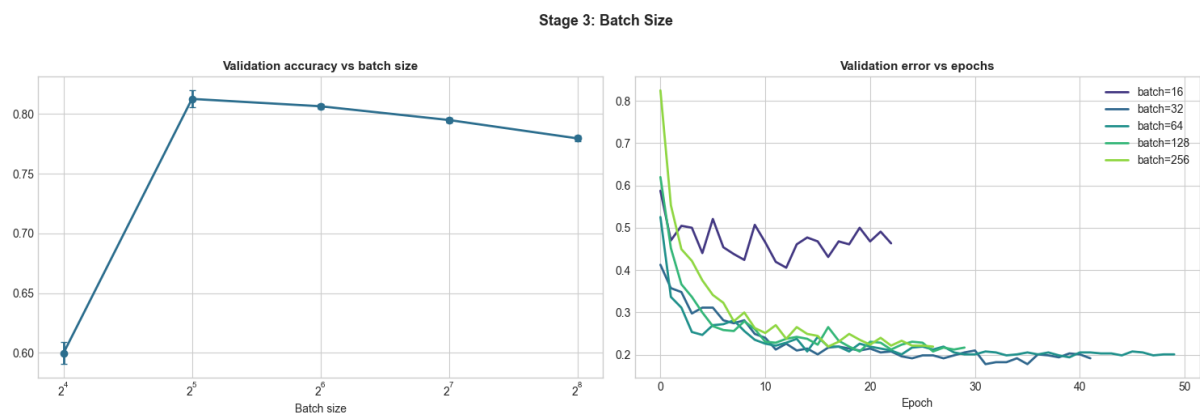


Figure 12: Stage 3: validation accuracy vs batch size and validation error vs epochs

Inference: Batch size 16 was unstable at this learning rate, with the validation accuracy stuck at 0.5998 and the training loss jumping between 1.2 and 2.1. Batch sizes 32 and 64 gave 0.8126 and 0.8065. Accuracy then fell for 128 and 256 (0.7949, 0.7796) because there are fewer updates per epoch. This was the hyperparameter with the biggest spread (about 21 points).

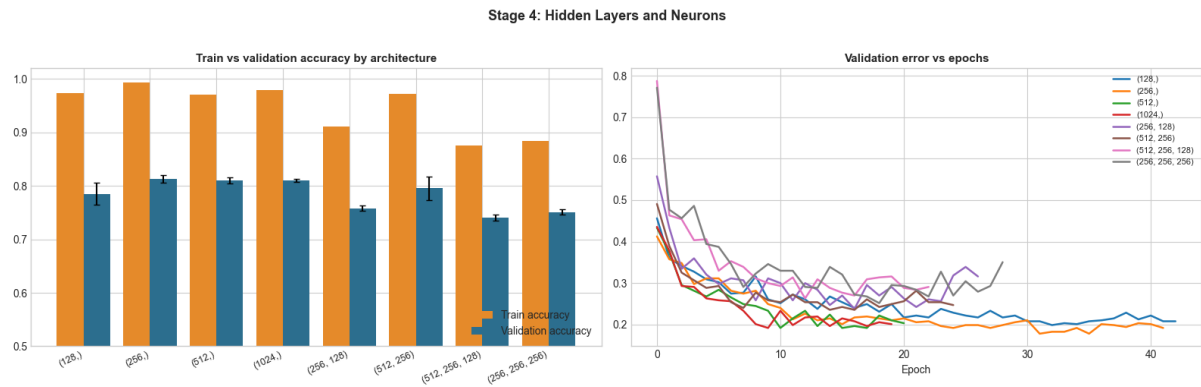


Figure 13: Stage 4: train vs validation accuracy for each architecture and validation error vs epochs

Inference: The single hidden layer of 256 units gave the best validation accuracy (0.8126), and 512 or 1,024 units gave about the same (0.8103). All deeper networks were worse, for example (512, 256, 128) reached only 0.7412. With just 2,294 training images the extra layers do not help, and they also stopped at a lower training accuracy (0.875 to 0.911).

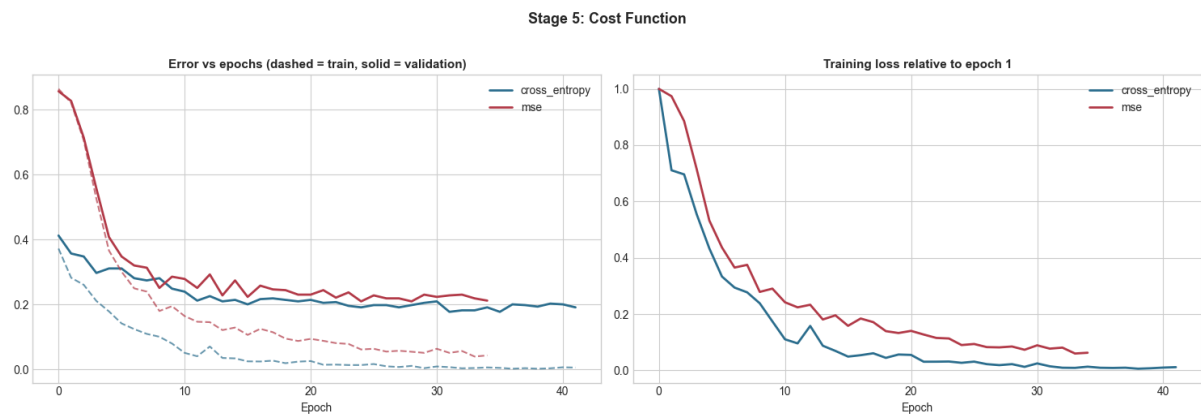


Figure 14: Stage 5: error and relative training loss for cross-entropy and MSE

Inference: Cross-entropy reached a validation accuracy of 0.8126 against 0.7911 for MSE. The MSE model also fitted the training data less (0.9393 against 0.9942). The gradient of the squared error through the softmax becomes small when the prediction is confidently wrong, which is one reason cross-entropy is preferred for classification.

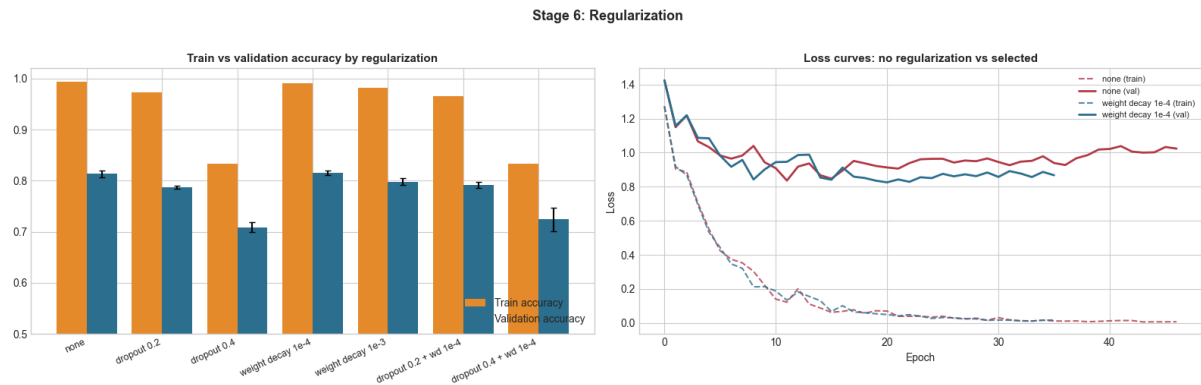


Figure 15: Stage 6: train vs validation accuracy for the regularization settings and loss curves with and without regularization

Inference: All settings have a large gap between training and validation accuracy (0.11 to 0.19). Dropout reduced the validation accuracy (0.7873 for 0.2 and 0.7089 for 0.4) because the network is small and dropout removed too much of it. Weight decay 10^{-4} gave the best result (0.8157), but only 0.2 points above no regularization, which is within the seed variation.

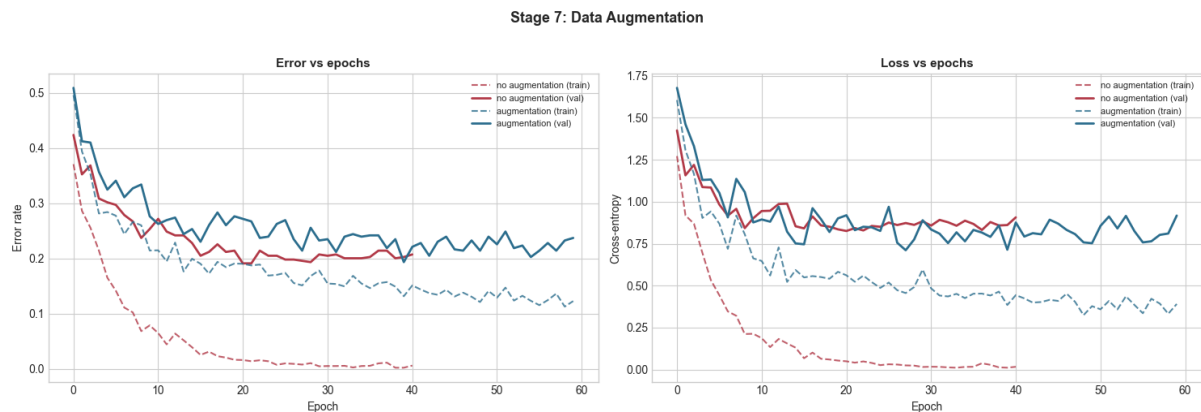


Figure 16: Stage 7: error and loss vs epochs with and without data augmentation

Inference: Augmentation reduced the train-validation gap from 0.1753 to 0.0584, so it does reduce overfitting. The validation accuracy did not improve (0.8049 against 0.8157), and training took almost twice as long. It was therefore not selected.

Final MLP Results

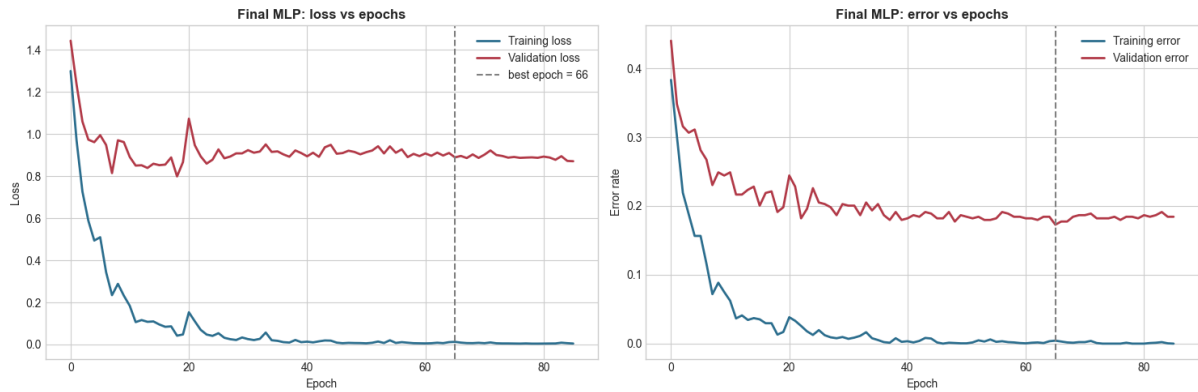


Figure 17: Final MLP loss vs epochs (left) and error vs epochs (right)

Inference: The training loss falls to about 0.004 and the training error reaches almost 0, while the validation loss is lowest near epoch 20 (0.87) and then rises slightly. The best validation error was at epoch 66 and training stopped at epoch 86 because of early stopping. This is a clear sign of overfitting. The final model has 0.9956 training accuracy and 0.8272 validation accuracy.

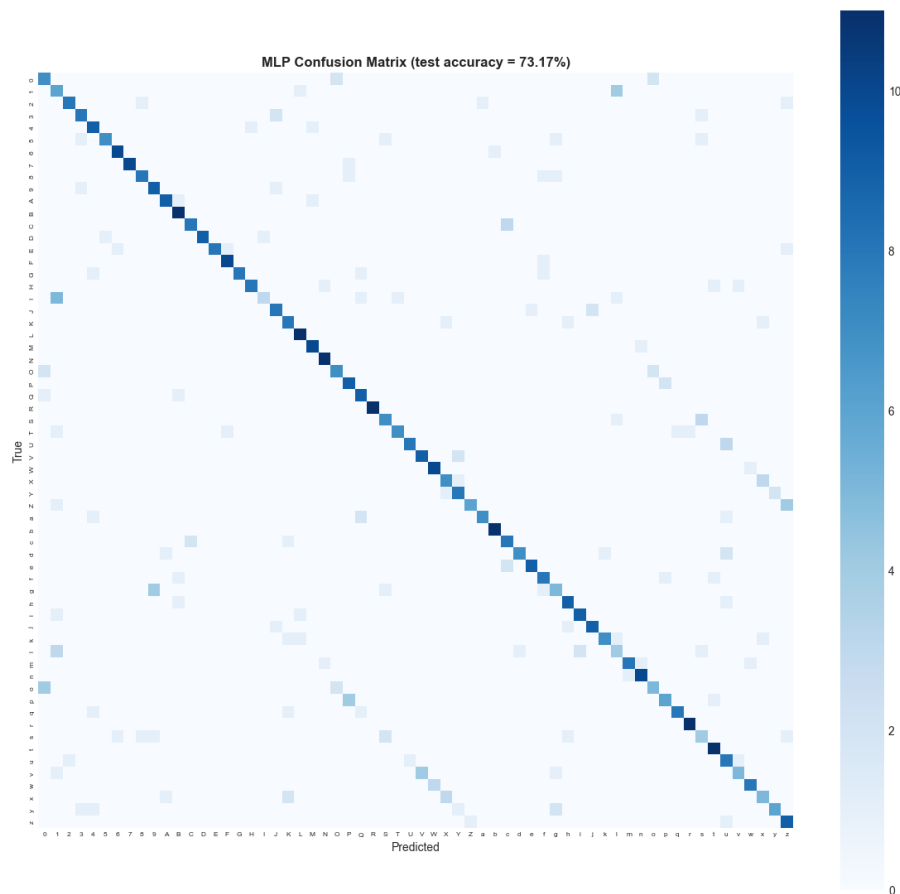


Figure 18: Confusion matrix of the tuned MLP on the test set (accuracy 73.17%)

Inference: The diagonal is stronger than for the PLA and there are fewer scattered

errors. The remaining mistakes are mainly case pairs and look-alikes such as I predicted as 1 (5 times), Z as z, v as V, p as P and g as 9. These characters are almost the same in a 32×32 image, so many of these errors are hard to remove.

A/B Comparison

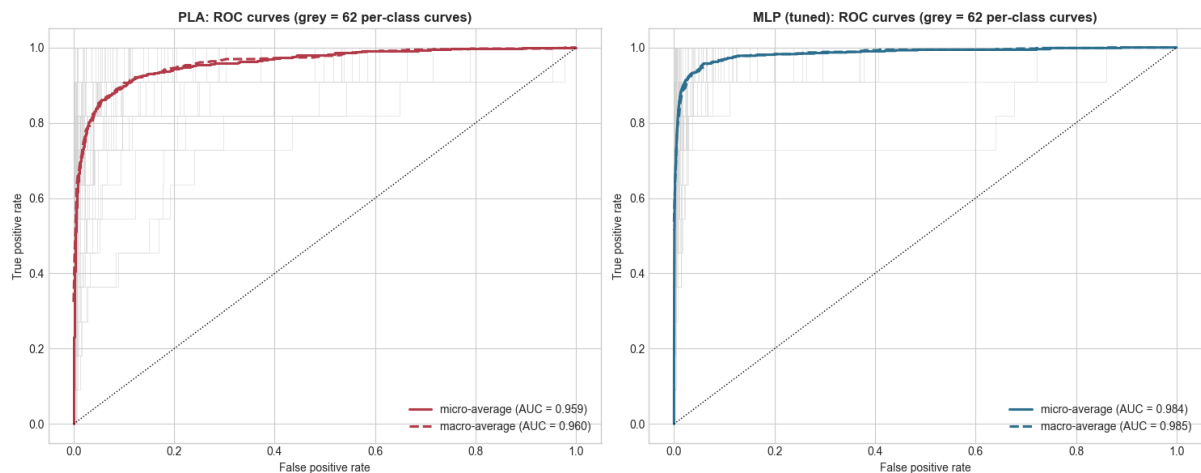


Figure 19: One-vs-rest ROC curves of PLA (left) and tuned MLP (right), with micro and macro averages (grey lines are the 62 class curves)

Inference: The MLP curves are closer to the top-left corner than the PLA curves. Micro AUC is 0.984 for the MLP and 0.959 for the PLA, and macro AUC is 0.985 against 0.960. The grey per-class curves are also more tightly grouped for the MLP. So the MLP ranks the correct class higher more consistently, even when its top prediction is wrong.

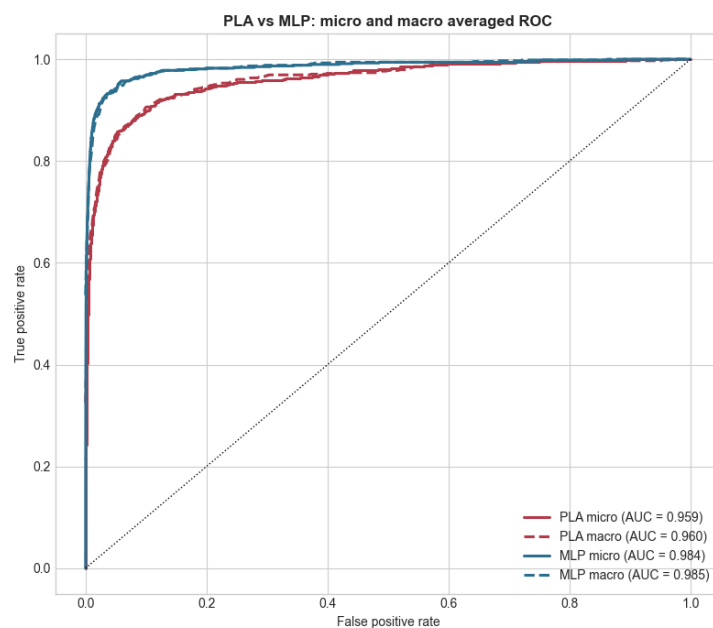


Figure 20: Micro and macro averaged ROC curves of PLA and MLP on the same axes

Inference: The MLP curves stay above the PLA curves over the whole range of false

positive rate. The gap is largest for false positive rates between about 0.02 and 0.3, where the MLP already has a true positive rate above 0.9 while the PLA is still around 0.85 to 0.95. Both models are well above the diagonal, but the MLP is clearly better.

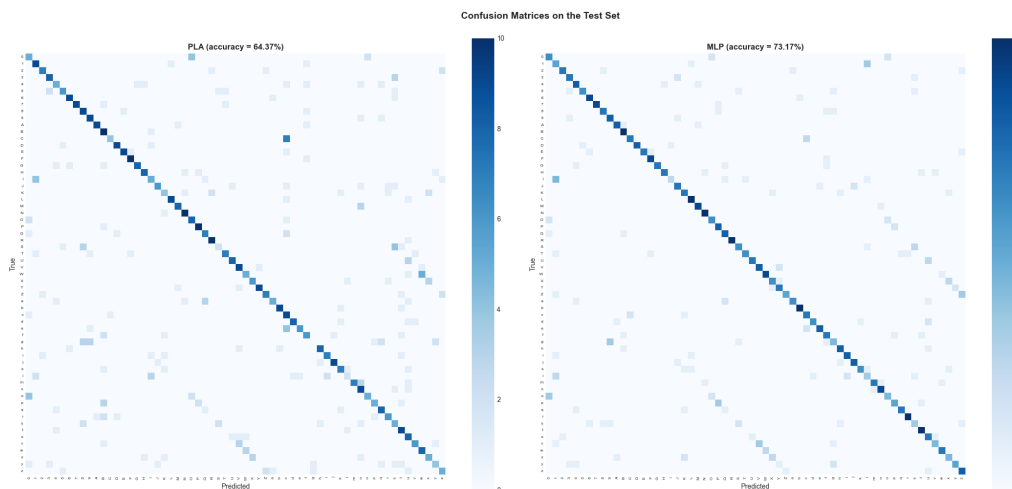


Figure 21: Confusion matrices of PLA (left) and MLP (right) on the test set

Inference: Placed side by side, the MLP matrix has a darker diagonal and fewer stray cells. Both models confuse the same kinds of characters, mostly upper and lowercase pairs and digits that look like letters. The MLP reduces these errors but does not remove them.

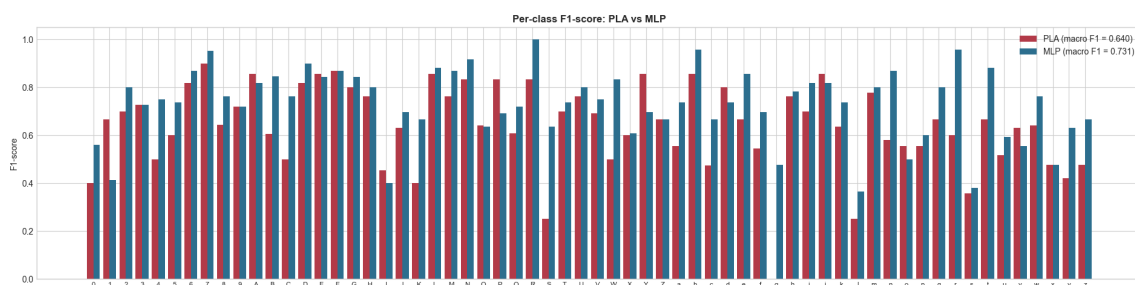


Figure 22: Per-class F1-score of PLA and MLP

Inference: The MLP has a higher F1 for most classes and macro F1 goes from 0.640 to 0.731. The biggest gains are for g, S, r and W, and the average F1 improves for all three groups (digits, uppercase, lowercase). A few classes such as I and 1 are actually worse for the MLP, and l, s, I, 1 and o stay low for both models.

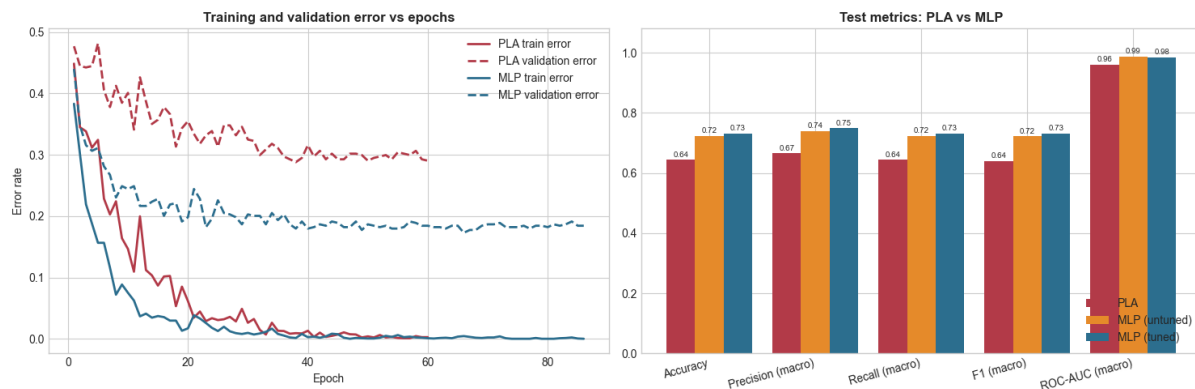


Figure 23: Training and validation error vs epochs for both models (left) and test metrics of PLA, untuned MLP and tuned MLP (right)

Inference: The PLA training error falls slowly and its validation error stays high, while the MLP validation error levels off much lower. On every metric the tuned MLP is better than the PLA. The untuned MLP is already close to the tuned one (72.29% against 73.17% accuracy), so most of the gain comes from having a hidden layer and only a small part from tuning.

11 Discussion

Why does PLA underperform? The PLA draws one straight line (hyperplane) per class in pixel space, and the 62 classes overlap a lot, so the update rule never settles (19 updates per epoch even after 60 epochs). It reaches 99.1% training accuracy but only 64.4% on the test set, so it memorises the training points without any margin. The learning rate does not help since it has no effect when the weights start at zero. The MLP hidden layer builds non-linear features and this gives 73.2% test accuracy.

Most important hyperparameters. Batch size and learning rate had the largest effect. Batch 16 gave 60.0% validation accuracy against 81.3% for batch 32, and the 12 optimizer and learning rate combinations ranged from 72.96% to 80.65%. The number of layers also mattered (a drop of about 7 points for three layers). Loss function, activation and weight decay had a small effect.

SGD vs Adam. The optimizer changed the convergence. Adam at 0.001 reached its best accuracy in about 22 epochs, while plain SGD at 0.01 took about 52 epochs. Adam was also less sensitive to the learning rate. SGD with momentum at 0.05 was the best overall and converged in about 29 epochs, so it was used in the final model.

More hidden layers. More layers did not always improve the results. Every deeper network was worse than the single 256-unit layer. With only 2,294 training images, extra layers add parameters that the data cannot support and make the optimisation harder. A flattened 32×32 image is a low-level input, so one hidden layer is enough.

Overfitting. The MLP overfits strongly: 99.6% training accuracy against 82.7% validation and 73.2% test accuracy, and the 5-fold estimate is $76.9\% \pm 1.9\%$. Early stopping and weight decay were used. Dropout did not help, and augmentation reduced the gap but did not raise the accuracy. Other ways to reduce it are more data per class, a convolutional network, or features such as HOG.

Strengths and limitations. The PLA is simple and trains in a few seconds but

cannot handle overlapping classes. The MLP is more accurate and gives better class scores, but it has more parameters (278,334 against 63,550), needs tuning and still overfits with 37 images per class. The test set has only 682 images, so a difference of about 1 point (like tuned against untuned) is within the normal variation.

12 Conclusion

A single-layer PLA and a tuned MLP were compared on the English Handwritten Characters dataset with 62 classes. The PLA reached 64.37% test accuracy (macro F1 0.640) and the tuned MLP reached 73.17% (macro F1 0.731), with macro ROC-AUC of 0.960 and 0.985. The final MLP uses one hidden layer of 256 ReLU units, SGD with momentum 0.9, learning rate 0.05, batch size 32, cross-entropy loss and weight decay 10^{-4} . Tuning mainly made training faster and more stable, and most of the improvement over the PLA came from the hidden layer. Both models mostly confuse look-alike characters, and both overfit, so the next steps would be more data or a convolutional network.

13 References

References

- [1] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [3] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.
- [5] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, 2019.
- [6] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [7] “English Handwritten Characters Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/dhruvildave/english-handwritten-characters-dataset>